



EuroCDP

EU Chips Design
Platform

IHP_COMM-3889

Nano SPI Device IP

Document Details

Deliverable title	IHP_COMM-3889 IP report
Date	24/09/2026
Lead beneficiary	CSIC
Type	IP Report
Authors	P. Ortega-Castro, M. Roales, and Macarena C. Martínez-Rodríguez (IMSE-CNM-CSIC)
Abstract	Report of Nano SPI Device IP.



Document Revision History

Version	Date	Description of change	Contributors
1.0	21/09/2026	First Version	IMSE-CNM-CSIC

Disclaimer

The information, documentation, and figures available in this deliverable are written by the “DECIDE: Democratizing European Chip Innovation and Design Ecosystem”, from now on referred to as EuroCDP, project consortium under EC grant agreement 101218811, and do not necessarily reflect the views of the European Commission.

The European Commission is not liable for any use that may be made of the information contained herein.

Copyright notice

© 2026 EuroCDP





Contents

1	Nano SPI Device IP	6
1.0.1	Overview and Origin	6
1.0.2	Functional Architecture	6
1.0.3	SPI Command Format	6
1.0.4	Internal Storage	7
1.0.5	SPI Interface	8
1.0.6	Protocol	8
1.0.7	Design and Implementation Flow	8
1.0.8	Physical Implementation Results	9
1.0.9	Verification Status	9
1.0.10	Maturity Assessment	9
1.0.11	Known Limitations	9
1.0.12	Conclusion	10



List of Figures

1	Serial Peripheral Interface (SPI) Interface, from: Digilent	7
2	SPI Simulation Test Result	9



List of Tables

1	SPI command encoding.	7
2	Internal storage target selection.	7
3	Main interfaces of the Nano SPI Device.	8
4	Physical implementation results.	9
5	Verification status.	10



Glossary

IP	Intellectual Property
PDK	Process Design Kit
RTL	Register-Transfer Logic
SPI	Serial Peripheral Interface

1 Nano SPI Device IP

1.0.1 Overview and Origin

The Nano SPI Device is a lightweight SPI slave peripheral designed to provide a simple interface for controlling output signals and accessing an internal 8-bit storage register. The device receives commands through an SPI interface and provides the corresponding response through the SPI MISO line.

The device was developed as a digital IP targeting the IHP 130 nm CMOS technology. The implementation was performed using the IHP 130 nm Process Design Kit (PDK).

This Intellectual Property (IP) is derived from the Apache-2.0-licensed ["SimpleSPIdev"](#). It provides an implementation of this peripheral communication interface in IHP 130nm PDK, performed using OpenLane Flow provided by Tiny Tapeout.

The configuration set by its author was ported to a regular LibreLane Flow in order to reproduce with the highest fidelity as possible the original output of the original Flow.

The current release targets the IHP 130nm CMOS PDK. In addition to the synthesizable Register-Transfer Logic (RTL) implementation, the release includes a placed-and-routed hard macro and the associated logical, physical, timing, and verification views required for integration into a larger design.

1.0.2 Functional Architecture

The Nano SPI Device operates as an SPI slave. The external controller provides the SPI clock, chip-select, and serial input data, while the device returns data through the MISO signal.

The implemented interface consists of four SPI signals:

- **MOSI**: serial data input from the SPI master;
- **SPI CLK**: serial clock generated by the SPI master;
- **SPI CS**: chip-select signal used to delimit an SPI transaction;
- **MISO**: serial data output from the SPI slave.

The SPI transaction is based on an 8-bit shift register. Incoming data is sampled on the rising edge of the SPI clock, while the next output bit is presented on the falling edge. After eight received bits, the command is considered complete and is subsequently decoded and executed by the internal control logic.

The general data path can be represented as:

SPI Master → MOSI → Command Decoder → Internal Registers

and, for read operations:

Internal Registers → MISO → SPI Master.

1.0.3 SPI Command Format

Each SPI command consists of eight bits. The four most significant bits select the operation, while the four least significant bits contain its argument.

$$\text{SPI_DATA}[7:0] = \{\text{CMD}[3:0], \text{ARG}[3:0]\}.$$

The command field is decoded using the following values:

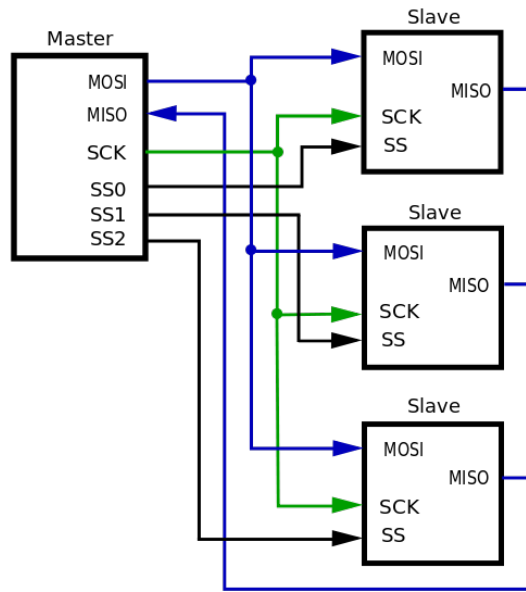


Figure 1: SPI Interface, from: Digilent

Table 1: SPI command encoding.

Command	CMD[3:0]	ARG[3:0]	Function
STATUS	0x0	Field	Returns the status associated with the selected field.
SET	0x1	IO pin	Activates the corresponding output function.
CLEAR	0x2	IO pin	Deactivates the corresponding output function.
TARGET	0x3	Target	Selects the storage nibble used by subsequent read/write operations.
WRITE	0x4	Data	Writes a four-bit value to the selected storage nibble.
READ	0x5	–	Returns the value stored in the selected storage nibble.

1.0.4 Internal Storage

The device includes an 8-bit internal storage register named `storage`. The register can be accessed through the `TARGET`, `WRITE`, and `READ` commands.

The `TARGET` command selects which half of the storage register is accessed by subsequent read and write operations.

Table 2: Internal storage target selection.

TARGET argument	Selected bits	Description
0x0	<code>storage[3:0]</code>	Selects the lower four bits of the internal storage register.
0x1	<code>storage[7:4]</code>	Selects the upper four bits of the internal storage register.

Therefore, the target selection can be represented as:

$$\text{TARGET} = 0 \Rightarrow \text{storage}[3:0]$$

`TARGET = 1 ⇒ storage[7:4].`

The remaining values of the four-bit target argument are not explicitly used by the current RTL implementation.

1.0.5 SPI Interface

The external interface of the top-level Nano SPI module is summarized in Table 3.

Table 3: Main interfaces of the Nano SPI Device.

Interface	Main signals	Description
Clock and reset	<code>clk, rst_n</code>	System clock and active-low reset.
SPI input	<code>ui_in[0], ui_in[1], ui_in[2]</code>	SPI MOSI, SPI clock, and chip-select signals, respectively.
SPI output	<code>uo_out[0]</code>	SPI MISO signal.
Clock monitor	<code>uo_out[2]</code>	Direct mirror of the system clock.
Status output	<code>uo_out[6]</code>	Status signal associated with the internal status LED.
Reset status	<code>uo_out[7]</code>	Indicates the reset state of the SPI device.
Debug output	<code>uo_out[1]</code>	Internal debug signal indicating SPI transaction activity.

The remaining dedicated input and output pins are unused. The bidirectional `uio[7:0]` interface is also unused in the current top-level implementation.

1.0.6 Protocol

The SPI protocol implemented by the device consists of 8-bit transactions. The chip-select signal delimits each transaction and must remain active while the eight clock cycles required to transfer one command are generated.

The transaction can be summarized as follows:

- **Chip Select:** The SPI master activates `CS` to start a transaction.
- **Command Transmission:** The master transmits an 8-bit word through MOSI. The four most significant bits identify the command and the four least significant bits provide its argument.
- **Data Reception:** The SPI slave samples the incoming data on the rising edge of the SPI clock.
- **Response:** The slave shifts the response through MISO. The output data is updated on the falling edge of the SPI clock.
- **Command Execution:** After eight received bits, the command is marked as complete and is subsequently decoded and executed.
- **Chip Select Release:** The master deactivates `CS`, completing the transaction.

1.0.7 Design and Implementation Flow

The IP was implemented using an automated RTL-to-GDS flow using LibreLane infrastructure library, covering logical verification, synthesis, physical implementation, timing analysis, physical verification, and release packaging.

The main stages were as follows:

1. RTL simulation using Cocotb;
2. logic synthesis using Yosys;
3. initial floorplanning and power-distribution planning;

4. standard-cell placement and optimization;
5. clock-tree synthesis;
6. global and detailed routing;
7. parasitic extraction;
8. multi-corner static timing analysis;
9. design-rule checking, layout-versus-schematic verification, and antenna checking;
10. packaging of logical, timing, physical, verification, and documentation views.

The implementation environment is summarized below:

- Flow: LibreLane;
- Process design kit: ihp-sg13g2;
- Standard-cell library: sg13g2_stdcell;
- Clock period constraint: 20 ns;
- Nominal target frequency: 50 MHz;
- Execution and testing platform: EuroCDP cloud server.

1.0.8 Physical Implementation Results

The Nano SPI Device was implemented using the IHP 130 nm PDK. The final physical implementation results are summarized in Table 4.

Table 4: Physical implementation results.

Metric	Result
Macro dimensions	167 × 108 μm
Clock constraint	20 ns
Setup violations	0
Hold violations	0
DRC violations	0
Antenna violations	0
LVS	Circuits match uniquely

1.0.9 Verification Status

The Nano SPI Device was verified at RTL and physical implementation levels. The main verification results are summarized in Table 5.

```
Test project behavior
test_spi.test_project passed
*****
** TEST                STATUS SIM TIME (ns) REAL TIME (s) RATIO (ns/s) **
*****
** test_spi.test_project    PASS      100000.00      0.00  45565497.01 **
*****
** TESTS=1 PASS=1 FAIL=0 SKIP=0      100000.00      0.00  30970272.47 **
*****
```

Figure 2: SPI Simulation Test Result

1.0.10 Maturity Assessment

The Nano SPI Device has been implemented using the IHP 130 nm technology.

[TO BE COMPLETED: Assign the final IP maturity level according to the project's maturity classification.]

1.0.11 Known Limitations

The following limitations apply to the current implementation:

- The current implementation supports a fixed 8-bit SPI transaction format consisting of a 4-bit command and a 4-bit argument.
- The current implementation does not expose a programmable bank of general-purpose output pins through the `SET` and `CLEAR` commands.
- The argument associated with the `SET` and `CLEAR` commands is not currently used to select different physical output pins.
- The `STATUS` command currently returns zero-valued status information.
- The `TARGET` command currently selects only between the lower and upper four-bit portions of the internal 8-bit `storage` register.
- The remaining target argument values are not assigned to additional registers or addresses in the current RTL.
- The bidirectional Tiny Tapeout `uio[7:0]` pins are not used by the current implementation.
- Detailed physical verification results are dependent on the final IHP 130 nm implementation and must be inserted once the corresponding reports are available.
- It must be added to `config.json` parameters like the PDK used, the Verilog files, design name and the equal die area as the tile size of Tiny Tapeout. For IHP has been necessary to change "RT_MAX_LAYER" from "met4" to "Metal4" because of the naming change between OpenLane and LibreLane (or between PDK versions).

1.0.12 Conclusion

The Nano SPI Device provides a compact SPI slave interface for command- based access to an internal 8-bit storage register and associated control signals. Its 8-bit command format divides each transaction into a 4-bit command field and a 4-bit argument field, allowing a simple communication protocol to be implemented.

The device supports `STATUS`, `SET`, `CLEAR`, `TARGET`, `WRITE`, and `READ` commands. The current RTL provides functional access to two four-bit portions of an internal 8-bit storage register through the `TARGET`, `WRITE`, and `READ` commands.

The design was implemented in Verilog and targeted to the IHP 130 nm CMOS technology. The top-level wrapper exposes the SPI interface through dedicated input and output pins, while additional outputs provide status, reset, debug, and system-clock monitoring signals.

Table 5: Verification status.

Verification activity	Status
RTL simulation	Passed
Synthesis checks	Passed
Post-synthesis gate-level simulation	Passed
Post-route gate-level simulation	Passed
Design-rule checking	Passed
Layout-versus-schematic verification	Passed
Multi-corner static timing analysis	Passed
Antenna verification	Passed
Lint	0 errors, 1 warning
Clock-domain crossing analysis	Not applicable
Logical equivalence checking / EQY	Not performed
Silicon validation	Not performed